AULA 09
PROGRAMAÇÃO ORIENTADA A OBJETOS

Programação em Python • Parte 1 de 2
Curso de Capacitação em Desenvolvimento Full Stack
Tema: Programação Orientada a Objetos • 10 horas
Professor: Msc. Hygo Sousa de Oliveira

(95) 99173-5676 / (92) 99171-0391
www.iteam.org.br
Av. Glaycon de Paiva nº 1820,
Mecejana
CEP: 69.304-560
Boa Vista/RR
CNPJ: 48.653.884/0001-00

Roteiro da aula

O que vamos ver nesta aula

1. Classes, objetos, atributos e métodos
2. Atributos de classe e de instância
3. Encapsulamento e @property
4. Herança, super() e sobrescrita
5. Polimorfismo e duck typing
6. Exemplos guiados e erros comuns

Objetivos de aprendizagem

Ao final da aula você será capaz de:

- **Criar classes, objetos, atributos e métodos**
  __init__, self e atributos de classe
- **Proteger dados com encapsulamento**
  Convenção _ e __, @property e validação
- **Reaproveitar código com herança**
  Classe base, super() e sobrescrita de métodos
- **Aplicar polimorfismo e duck typing**
  Mesmo método, comportamentos diferentes

Módulo 5 – Programação Orientada a Objetos

- Classes, objetos, atributos e métodos
- Encapsulamento e @property
- Herança e super()
- Polimorfismo e duck typing
- Métodos especiais e design de classes (Aula 10)

Classe, objeto, atributo e método

Quatro conceitos que sustentam toda a POO

Classe

Molde que define atributos e métodos. Ex.: `Carro`.

Objeto

Instância criada a partir da classe. Ex.: `Carro("Uno")`.

Atributo

Dado que o objeto guarda. Ex.: `modelo`, `km`.

Método

Função da classe que age sobre o objeto. Ex.: `rodar()`.

NOTA Analogia: a classe é a planta da casa; cada objeto é uma casa construída a partir dela, com seus próprios dados.

Criando uma classe e seus objetos

__init__ é o construtor e self é o próprio objeto

classe.py

```python
class Carro:
    def __init__(self, marca, modelo):
        self.marca = marca
        self.modelo = modelo
        self.km = 0

meu = Carro("Fiat", "Uno")
outro = Carro("VW", "Gol")
print(meu.marca, meu.modelo, meu.km)
print(outro.modelo)
print(type(meu).__name__)
```

SAÍDA

```
Fiat Uno 0
Gol
Carro
```

NOTA Cada objeto tem seus próprios atributos. `__init__` roda sozinho ao criar o objeto.

Métodos: o comportamento do objeto

Todo método de instância recebe self como primeiro parâmetro

metodos.py

```
class Carro:
    def __init__(self, modelo):
        self.modelo = modelo
        self.km = 0

    def rodar(self, distancia):
        self.km += distancia

    def painel(self):
        return f"{self.modelo}: {self.km} km"

c = Carro("Uno")
c.rodar(120)
c.rodar(30)
print(c.painel())
```

SAÍDA

```
Uno: 150 km
```

DICA Em `c.rodar(120)`, o Python passa `c` como `self` automaticamente.

Atributos de classe e de instância

Um é compartilhado por todos; o outro pertence a cada objeto

atributos.py

```
class Carro:
    rodas = 4
    total = 0

    def __init__(self, modelo):
        self.modelo = modelo
        Carro.total += 1

a = Carro("Uno")
b = Carro("Gol")
print(a.rodas, b.rodas, Carro.total)
b.modelo = "Polo"
print(a.modelo, b.modelo)
```

SAÍDA

```
4 4 2
Uno Polo
```

NOTA `rodas` e `total` são da **classe**. `modelo` é da **instância**: mudar em `b` não afeta `a`.

Encapsulamento: níveis de acesso

Em Python, a proteção é principalmente uma convenção

encapsulamento.py

```
class Cofre:
    def __init__(self):
        self.titular = "Ana"
        self._codigo = 1234
        self.__saldo = 500

c = Cofre()
print(c.titular, c._codigo)
print(hasattr(c, "__saldo"))
```

SAÍDA

```
Ana 1234
False
```

ATENÇÃO `_codigo` é protegido por convenção; `__saldo` é privado (o nome vira `_Cofre__saldo`).

@property: atributo calculado

Acessar um método como se fosse um atributo

property.py

```python
class Retangulo:
    def __init__(self, base, altura):
        self.base = base
        self.altura = altura

    @property
    def area(self):
        return self.base * self.altura

r = Retangulo(4, 3)
print(r.area)
r.base = 10
print(r.area)
```

SAÍDA

```
12
30
```

DICA `r.area` não leva parênteses e é recalculado a cada leitura.

@property com setter e validação

O setter protege o dado e recusa valores inválidos

produto.py

```python
class Produto:
    def __init__(self, preco):
        self.preco = preco

    @property
    def preco(self):
        return self._preco

    @preco.setter
    def preco(self, valor):
        if valor <= 0:
            raise ValueError("Preço inválido")
        self._preco = valor
```

uso

```python
p = Produto(2.5)
print(p.preco)
try:
    p.preco = -1
except ValueError as erro:
    print("Erro:", erro)
```

SAÍDA

```
2.5
Erro: Preço inválido
```

ATENÇÃO O valor real fica em `_preco`. No setter, nunca grave em `self.preco`: seria recursão infinita.

Herança: reaproveitando uma classe

A classe filha herda atributos e métodos da classe mãe

heranca.py

```python
class Animal:
    def __init__(self, nome):
        self.nome = nome

    def apresentar(self):
        return f"Sou {self.nome}"

class Cachorro(Animal):
    def latir(self):
        return "Au au!"

rex = Cachorro("Rex")
print(rex.apresentar())
print(rex.latir())
```

SAÍDA

```
Sou Rex
Au au!
```

NOTA `Cachorro(Animal)` herda tudo de `Animal` e acrescenta o que é só dele.

super() e sobrescrita de métodos

A filha estende o construtor e redefine comportamentos

animais.py

```python
class Animal:
    def __init__(self, nome):
        self.nome = nome

    def som(self):
        return "..."

class Gato(Animal):
    def __init__(self, nome, cor):
        super().__init__(nome)
        self.cor = cor

    def som(self):
        return "Miau!"
```

uso

```python
g = Gato("Mimi", "preto")
print(g.nome, g.cor)
print(g.som())
```

SAÍDA

```
Mimi preto
Miau!
```

ATENÇÃO `super().__init__(...)` executa o construtor da mãe. Sem ele, `nome` não seria criado.

Verificando tipos e a ordem de busca

isinstance, issubclass e a MRO

tipos.py

```
class Animal:
    pass

class Gato(Animal):
    pass

g = Gato()
print(isinstance(g, Animal), issubclass(Gato, Animal))
print([c.__name__ for c in Gato.__mro__])
```

SAÍDA

```
True True
['Gato', 'Animal', 'object']
```

NOTA A MRO é a ordem de busca dos métodos: classe, mães e `object`.

Polimorfismo

Mesmo método, comportamentos diferentes conforme o objeto

polimorfismo.py

```python
class Animal:
    def falar(self):
        return "..."

class Cachorro(Animal):
    def falar(self):
        return "Au au!"

class Gato(Animal):
    def falar(self):
        return "Miau!"

for bicho in [Cachorro(), Gato(), Animal()]:
    print(type(bicho).__name__, bicho.falar())
```

SAÍDA

```
Cachorro Au au!
Gato Miau!
Animal ...
```

DICA O laço não precisa saber o tipo: cada objeto responde do seu jeito.

Duck typing

Se tem o método, serve: sem herança e sem checar tipos

duck.py

```
class Pato:
    def falar(self):
        return "Quack!"

class Robo:
    def falar(self):
        return "Bip bop!"

def fazer_falar(coisa):
    print(coisa.falar())

fazer_falar(Pato())
fazer_falar(Robo())
```

SAÍDA

```
Quack!
Bip bop!
```

NOTA `Pato` e `Robo` não têm nenhuma relação, mas ambos funcionam em `fazer_falar`.

Exemplo guiado 1: playlist

Um objeto que guarda e gerencia uma lista

DESAFIO Crie `Playlist` com nome e lista de músicas, o método `adicionar` e o método `resumo`.

playlist.py

```python
class Playlist:
    def __init__(self, nome):
        self.nome = nome
        self.musicas = []

    def adicionar(self, musica):
        self.musicas.append(musica)

    def resumo(self):
        n = len(self.musicas)
        return f"{self.nome}: {n} músicas"
```

uso

```python
p = Playlist("Estudo")
p.adicionar("Lo-fi 1")
p.adicionar("Lo-fi 2")
print(p.resumo())
print(p.musicas)
```

SAÍDA

```
Estudo: 2 músicas
['Lo-fi 1', 'Lo-fi 2']
```

Exemplo guiado 2: temperatura

Property calculada com setter que converte de volta

DESAFIO `Temperatura` guarda graus Celsius e expõe `fahrenheit` como property, também com setter.

temperatura.py

```python
class Temperatura:
    def __init__(self, celsius):
        self.celsius = celsius

    @property
    def fahrenheit(self):
        return self.celsius * 9 / 5 + 32

    @fahrenheit.setter
    def fahrenheit(self, valor):
        self.celsius = (valor - 32) * 5 / 9
```

uso

```python
t = Temperatura(25)
print(t.fahrenheit)
t.fahrenheit = 212
print(t.celsius)
```

SAÍDA

```
77.0
100.0
```

Exemplo guiado 3: personagens de um jogo

Herança, super() e polimorfismo juntos

DESAFIO Todo personagem ataca com 5; o guerreiro soma 10 ao ataque base; o mago ataca com 25.

jogo.py

```
class Personagem:
    def __init__(self, nome):
        self.nome = nome

    def atacar(self):
        return 5

class Guerreiro(Personagem):
    def atacar(self):
        return super().atacar() + 10

class Mago(Personagem):
    def atacar(self):
        return 25
```

uso

```
a = Guerreiro("Thor")
b = Mago("Merlin")
c = Personagem("Aldeão")
for p in (a, b, c):
    print(p.nome, p.atacar())
```

SAÍDA

```
Thor 15
Merlin 25
Aldeão 5
```

Erros comuns em POO

Reconhecer o sintoma acelera a correção

Sintoma	Causa provável	Como corrigir
TypeError: takes 1 positional argument but 2 were given	Faltou `self` na definição do método	Todo método de instância começa com `self`
AttributeError	Atributo não criado ou nome errado	Crie os atributos no `__init__` e confira a grafia
Atributos da mãe ausentes	A filha não chamou o construtor da mãe	Use `super().__init__(...)`
RecursionError no setter	O setter grava em `self.preco`	Guarde em `self._preco`
Lista compartilhada entre objetos	`itens = []` no corpo da classe	Crie a lista dentro do `__init__`

Atividades da Aula 9

Módulo 5: exercícios 01 a 04

- **Exercício 01 – Classes e Objetos**
  Sistema de cadastro de alunos
- **Exercício 02 – Encapsulamento**
  Conta bancária com @property
- **Exercício 03 – Herança e super()**
  Hierarquia Funcionario → Gerente / Estagiario
- **Exercício 04 – Polimorfismo e Duck Typing**
  Canais de notificação: Email, SMS e Push

GIT

Enunciados, instruções de execução e de entrega:
github.com/Hygo/python-fullstack-iteam-2sem

Obrigado(a)!

Próxima aula: Métodos especiais, design de classes e projetos finais
ITEAM – Instituto Tecnológico Educacional da Amazônia